

并行程序设计实验报告

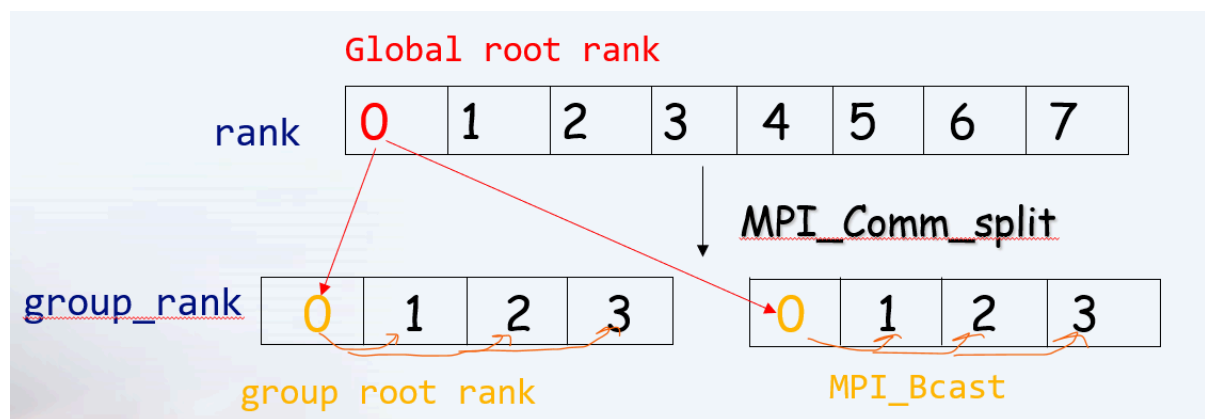
刘天润SC24219058

MPI-LAB

1 problem a

1.1) 写个将 MPI 进程按其所在节点分组的程序; (1.2) 在 1.1 的基础上, 写个广播程序, 主要思想是: 按节点分组后, 广播的 root 进程将消息“发送”给各组的“0 号”, 再由这些“0”号进程在其小组内执行 MPI_Bcast。

解答过程的抽象如下:



(1.1) 进程分组

实验环境只有一个节点, 在此直接使用 MPI_Comm_split 创建新的通信组。MPI_Comm_split 的用法如下: MPI_Comm_split(MyWorld, Color, Key, &SplitWorld) 函数调用则在通信域 MyWorld 的基础上产生了几何分割的子通信域。原通信域 MyWorld 中的进程按照不同的 Color 值处在不同的分割通信域中, 每个进程在不同分割通信域中的进程编号则由 Key 值来标识。

Rank in MyWorld	0	1	2	3	4	5	6	7	8	9
Color	0	1	2	0	1	2	0	1	2	0
Key	0	0	0	1	1	1	2	2	2	3
Rank in SplitWorld(Color=0)	0			1			2			3
Rank in SplitWorld(Color=1)		0			1			2		
Rank in SplitWorld(Color=2)			0			1			2	

```
#define GROUP_SIZE 3 // 每组的大小
int num_groups = (size + GROUP_SIZE - 1) / GROUP_SIZE; // 计算组数 (向上取整)

// 确定当前进程所属的组和组内的 rank
int group_id = rank / GROUP_SIZE;
int intra_group_rank = rank % GROUP_SIZE;
// 创建组内通信子
MPI_Comm group_comm;
MPI_Comm_split(MPI_COMM_WORLD, group_id, rank, &group_comm);
```

(1.2) 分层广播

```
// 非阻塞发送消息到每组的 root
for (int i = 0; i < num_groups; i++) {
    int group_root_rank = i * GROUP_SIZE; // 每组的 root (组内 rank 0) 在
    if (group_root_rank < size) { // 确保组存在
        MPI_Request request;
        MPI_Isend(message, sizeof(message), MPI_CHAR, group_root_rank, 0, &request); // 释放请求, 非阻塞发送不会阻塞全局 root
    }
}
// 各组的 root (组内 rank 0) 接收消息
if (intra_group_rank == 0) {
    MPI_Recv(message, sizeof(message), MPI_CHAR, global_bcast_root, 0, MPI_COMM_WORLD);
    printf("Group %d root (rank %d in MPI_COMM_WORLD) received message: %s\n", group_id, rank, message);
}

// 组内广播消息
MPI_Bcast(message, sizeof(message), MPI_CHAR, 0, group_comm);
```

运行结果如下:

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pa> mpiexec pa-1
Global root (rank 0) broadcasting message to group roots...
Group 0 root (rank 0 in MPI_COMM_WORLD) received message: Hello from rank 0!
Rank 0 (group 0, intra-group rank 0) received message: Hello from rank 0!
Group 2 root (rank 6 in MPI_COMM_WORLD) received message: Hello from rank 0!
Rank 6 (group 2, intra-group rank 0) received message: Hello from rank 0!
Rank 1 (group 0, intra-group rank 1) received message: Hello from rank 0!
Group 1 root (rank 3 in MPI_COMM_WORLD) received message: Hello from rank 0!
Rank 3 (group 1, intra-group rank 0) received message: Hello from rank 0!
Rank 4 (group 1, intra-group rank 1) received message: Hello from rank 0!
Rank 5 (group 1, intra-group rank 2) received message: Hello from rank 0!
Rank 2 (group 0, intra-group rank 2) received message: Hello from rank 0!
Rank 7 (group 2, intra-group rank 1) received message: Hello from rank 0!
```

注意这里要用 **非阻塞发送 (MPI_Isend)** , 否则会造成死锁:

```

job aborted:
[ranks] message

[0] fatal error
Fatal error in MPI_Send: Other MPI error, error stack:
MPI_Send(buf=0x00000000000061FDC0, count=50, MPI_CHAR, dest=0, tag=0, MPI_COMM_WORLD) failed
DEADLOCK: attempting to send a message to the local process without a prior matching receive

[1-7] terminated

---- error analysis ----

```

原因分析：全局 root 进程 (rank 0) 使用 MPI_Send 向每组的 root 发送消息，但是如果接收方 (组内的 rank 0) 没有准备好调用 MPI_Recv，发送方会阻塞等待，从而导致死锁。使用 MPI_Isend 替代阻塞的 MPI_Send，并释放请求对象 (MPI_Request_free)，可以避免全局 root 进程等待接收方就绪的情况下发生死锁。(但是不让rank0给rank0发消息虽然不会死锁，但程序也会运行不了?)：

```

PS E:\graduate1stCourse\parallel\repo\mpi-lab\pa> mpiexec pa-1
mpiexec aborting job...

```

```

job aborted:
[ranks] message

[0] job terminated by the user

[1-7] terminated

---- error analysis ----

[0] on LTR_LAPTOP

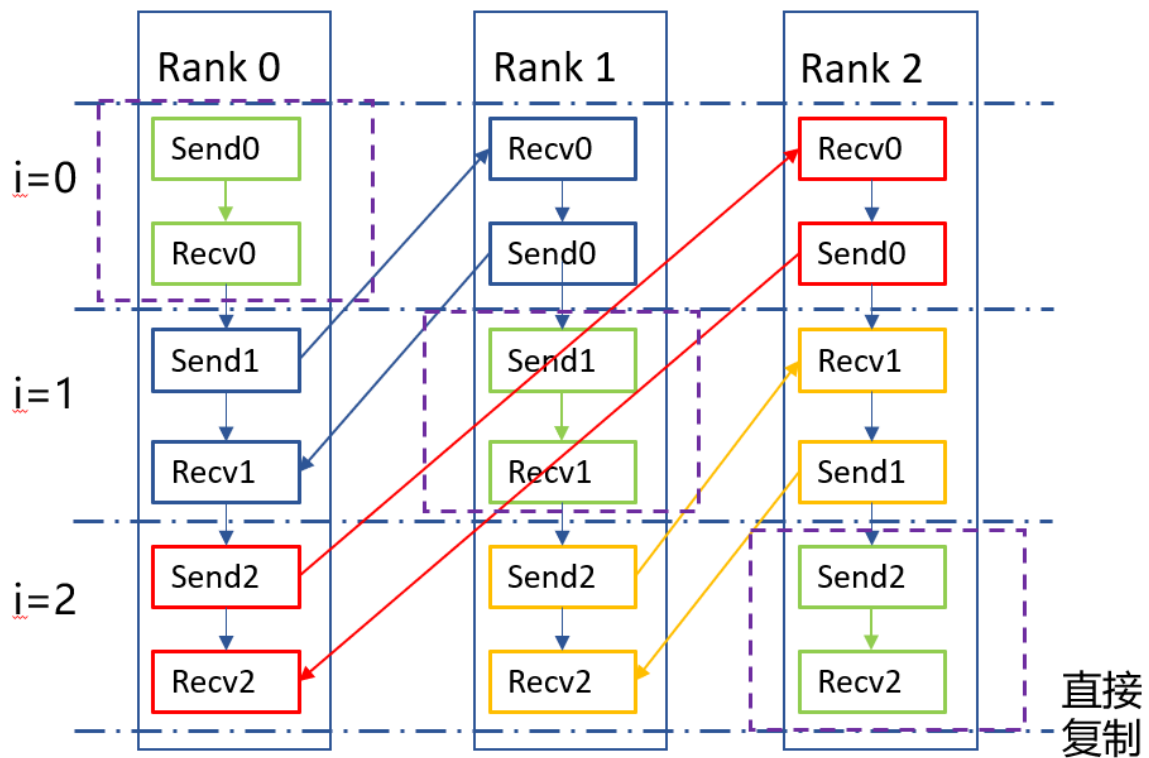
```

2 problem b

使用 MPI_Send 和 MPI_Recv 来模拟 MPI_Alltoall。将你的实验与相关 MPI通信函数做评测和对比。

方法一

第一种自定义的MPI_Alltoall函数，算法抽象如下：



每个进程都会遍历所有进程（包括自己），如果当前进程 i 不是自己 ($i \neq \text{rank}$)，则使用 MPI_Send 发送数据到进程 i ，并使用 MPI_Recv 从进程 i 接收数据。

对比自定义的Alltoall函数和MPI_Alltoall的性能，运行结果如下：

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pb> mpiexec pb -n 2
my alltoall time: 0.003642
MPI_alltoall total time: 0.000179
```

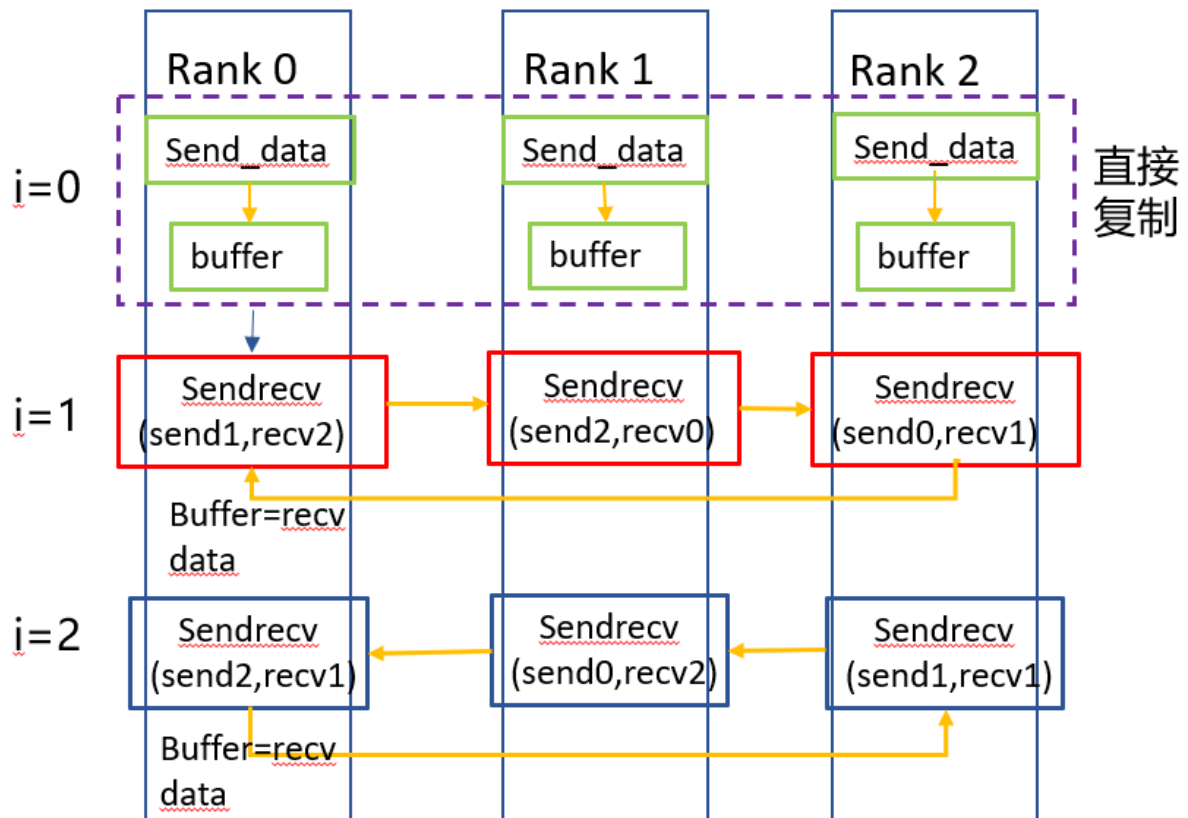
设置不同的进程数多次实验，得到MPI和自定义alltoall的时间花费如下表：

进程数	2	4	6	8
my	0.003820	0.008050	0.005132	0.007585
mpi	0.000192	0.000271	0.000095	0.000108
时间比	19.90	29.7	54.0	70.2

由上表可得，用MPI在进程间进行多对多通信时，直接用MPI_Send和MPI_Recv函数虽能实现基本功能，但耗费的时间非常多，效率显著低于MPI自带的方法MPI_Alltoall。且随着互相通信的进程数增加，二者的差距会越来越大。这是因为循环进程逐个send和recv的基础方法不能充分利用多个进程，存在进程空转等待接收较长的情况，增大了时延。

方法二

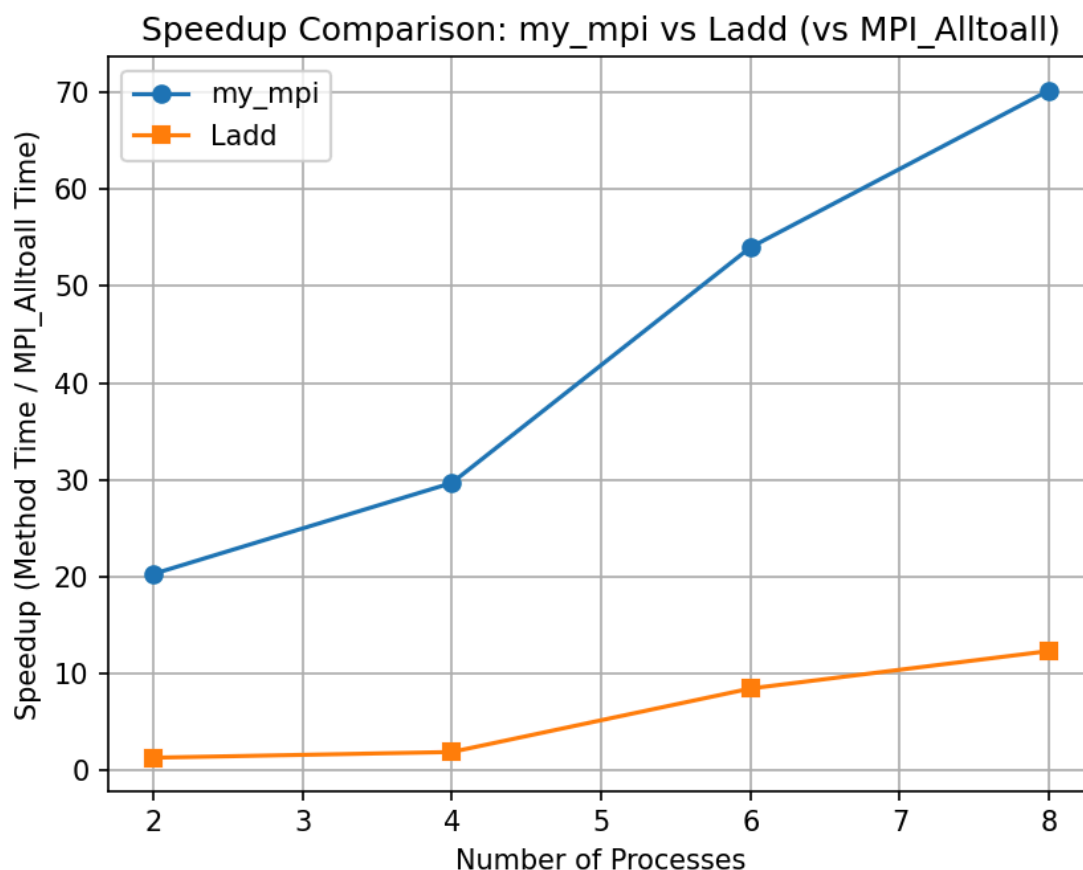
可采用 **循环发送** 接收的方式对其进行并行优化，这种方法使所有处理器每个步骤都可以发送接收数据，不需要等待接收数据。



运行结果如下:

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pb> mpiexec pb-2
Rank 6 received data: 20 21 22 23 24 25 26 27
Rank 0 received data: 40 41 42 43 44 45 46 47
Tony Ladd Alltoall execution time: 0.001332 seconds
Rank 5 received data: 10 11 12 13 14 15 16 17
Rank 4 received data: 0 1 2 3 4 5 6 7
Rank 2 received data: 60 61 62 63 64 65 66 67
Rank 3 received data: 70 71 72 73 74 75 76 77
Rank 7 received data: 30 31 32 33 34 35 36 37
Rank 1 received data: 50 51 52 53 54 55 56 57
```

设置不同进程数，画出性能比较图如下:



由图，优化后的方法二可以显著降低全局通信alltoall的时间，比第一种方法提升了近10倍。方法二在进程数小于等于4时性能逼近MPI_Alltoall函数，当进程数较大时和MPI_Alltoall仍有较大差距。

3 problem c

N 个处理器求 N 个数的全和，要求每个处理器均保持全和。

(1)

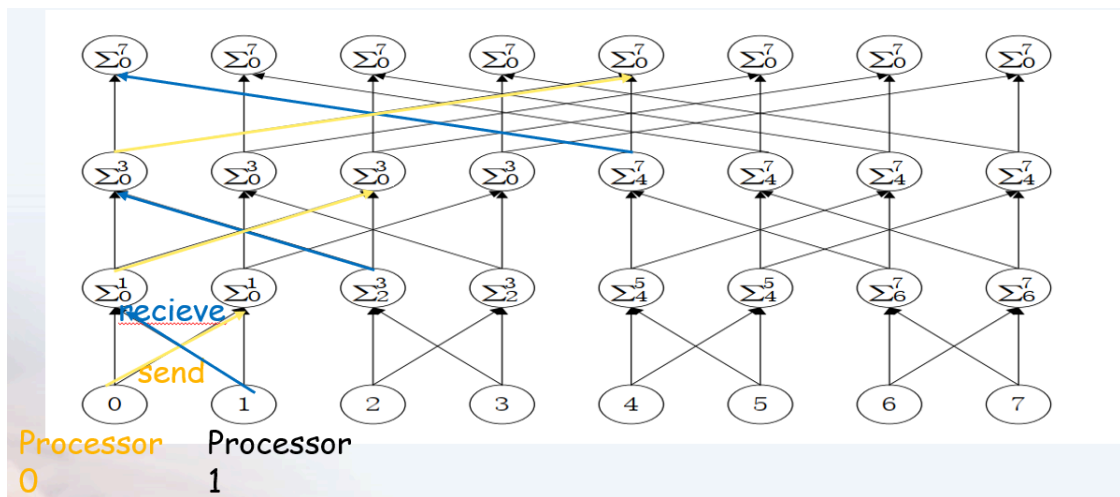
给出蝶式全和计算的 MPI 程序实现（设 N 为 2 的幂次方）。

运行结果如下，每个处理器均得到总和28。

partner in step 0:3	partner in step 0:7
partner in step 1:0	partner in step 1:4
partner in step 2:6	partner in step 2:2
Processor 2 has total sum = 28	Processor 6 has total sum = 28
partner in step 0:0	partner in step 0:6
partner in step 1:3	partner in step 1:5
partner in step 2:5	partner in step 2:3
Processor 1 has total sum = 28	Processor 7 has total sum = 28
partner in step 0:1	partner in step 0:5
partner in step 1:2	partner in step 1:6
partner in step 2:4	partner in step 2:0
Processor 0 has total sum = 28	Processor 4 has total sum = 28
partner in step 0:2	partner in step 0:4
partner in step 1:1	partner in step 1:7
partner in step 2:7	partner in step 2:1
Processor 3 has total sum = 28	Processor 5 has total sum = 28

解题说明:

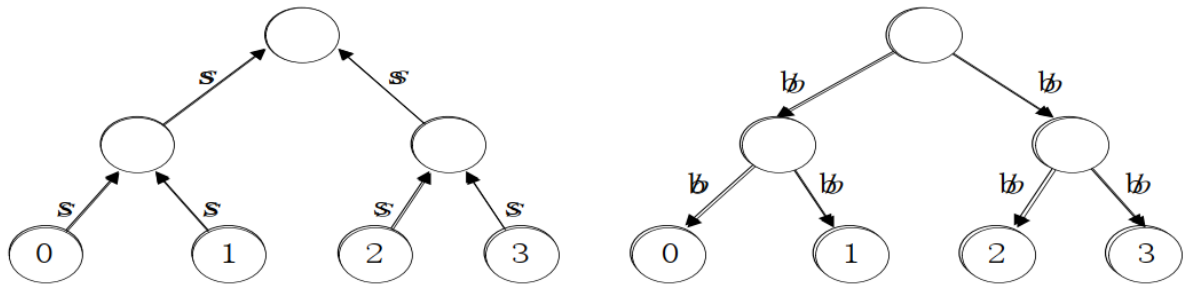
1. 使用MPI_Sendrecv函数，每个处理器都发送消息给partner并接受来自partner的消息。下图以处理器processor 0为例，描述处理器在整个过程中的消息传递操作。



2. 计算通信的处理器partner: 根据蝶形运算的规律，编号相差 2^{step} 的处理器会进行通信。可以通过异或操作实现，每个处理器分别跟0001, 0010, 0100异或。例如第1个处理器0001，异或的结果是0, 0011, 0101，即0, 3, 5。

(2)

(2) 二叉树方式求全和示意图如下：需要 $2\log N$ 步。



给出二叉树方式全和计算的 **MPI** 程序实现。

```
// 假设每个处理器持有的初始值是其 rank
local_value = rank;

// 向上汇聚阶段 (LogN 步)
for (step = 1; step < size; step *= 2) {
    if (rank % (2 * step) == 0) {
        // 当前节点收集来自子节点的数据
        MPI_Recv(&received_value, 1, MPI_INT, rank + step, 0, MPI_COMM_WORLD);
        local_value += received_value;
    } else if (rank % step == 0) {
        // 当前节点将数据发送给父节点
        MPI_Send(&local_value, 1, MPI_INT, rank - step, 0, MPI_COMM_WORLD);
        break; // 子节点任务完成, 退出
    }
}

// 向下广播阶段 (LogN 步)
for (step = size / 2; step >= 1; step /= 2) {
    if (rank % (2 * step) == 0) {
        // 当前节点向子节点发送数据
        MPI_Send(&local_value, 1, MPI_INT, rank + step, 0, MPI_COMM_WORLD);
    } else if (rank % step == 0) {
        // 当前节点接收来自父节点的数据
        MPI_Recv(&local_value, 1, MPI_INT, rank - step, 0, MPI_COMM_WORLD);
    }
}
```

运行结果


```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pc> mpiexec pc_2
Processor 5 has total sum = 28
Processor 1 has total sum = 28
Processor 2 has total sum = 28
Processor 4 has total sum = 28
Processor 0 has total sum = 28
Processor 3 has total sum = 28
Processor 7 has total sum = 28
Processor 6 has total sum = 28
```

解题说明:

1. 由题图，二叉树求和的方法：在每一步选择一个处理器储存求和的结果，向上传递汇总到一个处理器，再由这个处理器将求和结果依次传递给剩余处理器。两个过程分别需要 $\log N$ 步，故一共 $2\log N$ 。
2. 与碟式运算相比，除了第一步所有处理器发送数据外，不是所有处理器参与计算。处理器每次也不是和同一个处理器通信，而是接受子节点数据并发送数据给父节点处理器，因此需要分别使用MPI_Send和MPI_Recv通信。

4 problem d

V-3 矩阵相乘的另一种并行算法是 **Fox 算法** (Fox Algorithm): 将待相乘的矩阵 A 和 B 分成 p 个方块 $A_{i,j}$ 和 $B_{i,j}$ ($0 \leq i, j \leq \sqrt{p} - 1$), 每块大小为 $(n/\sqrt{p}) \times (n/\sqrt{p})$, 并将它们分配给 $\sqrt{p} \times \sqrt{p}$ 个处理器 ($P_{0,0}, P_{0,1}, \dots, P_{\sqrt{p}-1, \sqrt{p}-1}$)。开始时处理器 $P_{i,j}$ 存放有块 $A_{i,j}$ 和 $B_{i,j}$, 并负责计算块 $C_{i,j}$ 。然后 Fox 算法执行以下各步 $\sqrt{p}$ 次迭代, 即可完成:

- ① 选中对角块 $A_{i,i}$ 并将其向所在行的 $\sqrt{p} - 1$ 个处理器进行一到多播送;
- ② 各处理器将所收到的 A 阵的块与 B 阵原有的块进行乘-加运算;
- ③ B 阵的块向上循环 1 步;
- ④ 如果 $A_{i,j}$ 是本次播送的块, 则下次应选块 $A_{i, (j+1) \bmod \sqrt{p}}$ 向同行的 $\sqrt{p} - 1$ 个处理器播送, 然后转第②步。

请写出 Fox 算法的并行程序, 并用适当的编译环境调试运行。

矩阵分块后, 可以按照高等代数里的知识计算

$$\begin{aligned} C_{ij} &= \sum_{k=0}^{q-1} A_{ik} B_{kj} \\ &= A_{i0} B_{0j} + A_{i1} B_{1j} + \dots + A_{i, q-1} B_{q-1, j} \end{aligned}$$

但分析上述过程, 会发生 **数据访问冲突**: 如下图所示, 在进行第一次的计算时, 如果按行优先的计算原则, 那么 A_{00} 必然会 R-R 冲突, 而 B 矩阵的读取不会发生冲突。同理对角线上的矩阵块都存在访问冲突, 因此规避 A 子矩阵的冲突时必然要做的事情, fox 算法就是用行通信子将 A_{00} 广播给每一行的 C_{0j} 来解决此问题。

A ₀₀	A ₀₁	A ₀₂	A ₀₃
A ₁₀	A ₁₁	A ₁₂	A ₁₃
A ₂₀	A ₂₁	A ₂₂	A ₂₃
A ₃₀	A ₃₁	A ₃₂	A ₃₃

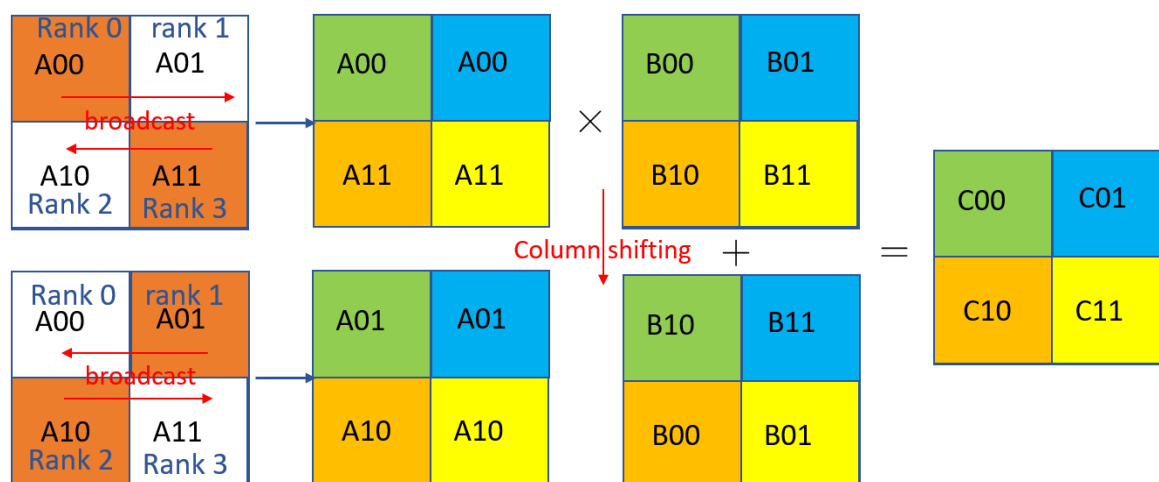
×

B ₀₀	B ₀₁	B ₀₂	B ₀₃
B ₁₀	B ₁₁	B ₁₂	B ₁₃
B ₂₀	B ₂₁	B ₂₂	B ₂₃
B ₃₀	B ₃₁	B ₃₂	B ₃₃

=

C ₀₀	C ₀₁	C ₀₂	C ₀₃
C ₁₀	C ₁₁	C ₁₂	C ₁₃
C ₂₀	C ₂₁	C ₂₂	C ₂₃
C ₃₀	C ₃₁	C ₃₂	C ₃₃

根据题述fox算法的原理，可视化计算过程如下（由于我的电脑CPU只有8线程，下以分块数量block=4为例）：



如上图，不同色块表示不同线程/处理器，处理器同时存储划分的子块 A_{ij} 和 B_{ij} 。相乘表示所有处理器同时计算 $A_{ij} \times B_{ij}$ ，并存储点乘的结果 C_{ij} 。

受江胜同学分享的启发，可以在每一行线程和每一列线程分别划分子通信域，在子通信域内进行通信，方便代码编写，提升执行效率。

运行结果如下：

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pd> mpiexec -n 4 pd
Process:1
68      74
196     218
Process:0
56      62
152     174
Running time:0.011000s
Process:3
324     362
452     506
Process:2
248     286
344     398
```

和以下串行执行的结果对比可知，计算结果正确。

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pd> pd_serial
```

Matrix a:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Matrix b:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

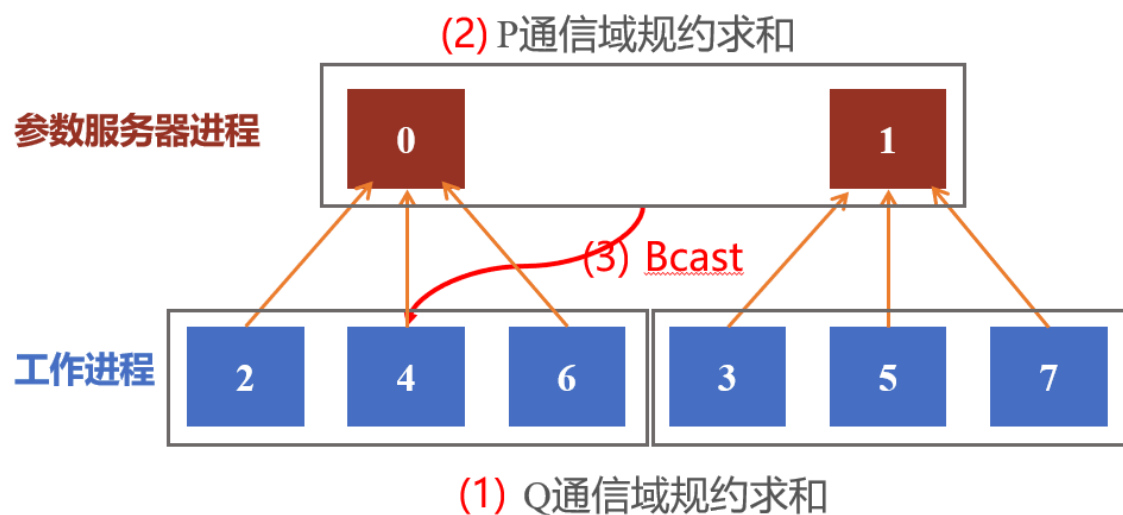
Matrix a*b:

56	62	68	74
152	174	196	218
248	286	324	362
344	398	452	506

Running time:0.006000s

5 problem e

根据题述用MPI实现互动过程，算法的抽象如下。我的CPU共有8个线程，设置前2个线程为参数处理器，剩下的6个为工作处理器。



```

int main( int argc, char* argv[] ){
    int P=2; //2个参数服务器进程
    int Q =6; //6个工作进程
    int rank, size;
    int senddate=0,sum=0,allsum=0;
    double average=0.0;
    double collect[10];
    int count=2; //展示两轮互动过程

    MPI_Init( &argc, &argv );
    MPI_Status(status);
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );

    MPI_Comm Pcomm,Qcomm;
    //参数服务器组成子通信域Pcomm
    MPI_Comm_split(MPI_COMM_WORLD,rank/P,rank%P,&Pcomm);
    //每个参数服务器与其对应的所有工作进程组成一个子通信域Qcomm
    MPI_Comm_split(MPI_COMM_WORLD,rank%P,rank/P,&Qcomm);
    //注意初始化随机数种子。time(NULL) 获取当前时间, rank 确保每个进程有不同的种子
    srand(time(NULL) + rank);
    for(int i=0;i<count;i++){
        if(rank>P-1){
            senddate=rand()%100;
            printf("process%d random%d: %d\n",rank,i+1,senddate);
        }
        //参数服务器对其对应工作进程产生的随机数进行求和归约
        MPI_Reduce(&senddate,&sum,1,MPI_INT,MPI_SUM,0,Qcomm);
        //参数服务器间求和归约 (每个都持有全和)
        MPI_Allreduce(&sum,&allsum,1,MPI_INT,MPI_SUM,Pcomm);
        average=(double)allsum/Q; //注意此处求均值是除以工作进程的数量
        //参数服务器将平均值广播给对应的工作进程
        MPI_Bcast(&average,1,MPI_DOUBLE,0,Qcomm);
        collect[i]=average;
    }
}

```

运行结果如下:

```

PS E:\graduate1stCourse\parallel\repo\mpi-lab\pe> mpiexec pe
process5 random1: 39
process5 random2: 35
process:5 average number:      37.000000      49.333333
process3 random1: 32
process3 random2: 38
process:3 average number:      37.000000      49.333333
process6 random1: 42
process6 random2: 15
process:6 average number:      37.000000      49.333333
process7 random1: 45
process7 random2: 64
process:7 average number:      37.000000      49.333333
process2 random1: 29
process2 random2: 58
process:2 average number:      37.000000      49.333333
process:0 average number:      37.000000      49.333333
process4 random1: 35
process4 random2: 86
process:4 average number:      37.000000      49.333333
process:1 average number:      37.000000      49.333333

```

如图，每个进程都获得了所有工作进程的平均值，以第一轮为例，程序计算的结果为37，由 $\text{average} = (39 + 32 + 42 + 45 + 29 + 35) / 6 = 37$ 可知计算正确。

6 problem f

矩阵 **A** 和 **B** 均为 $N \times N$ 的双精度数矩阵，有 **P** 个处理器。针对以下程序片段，分别采用按行块连续划分以及棋盘式划分方式，给出相应的 **MPI** 并行实现。

```
for(i=1; i<N-1; i++)
```

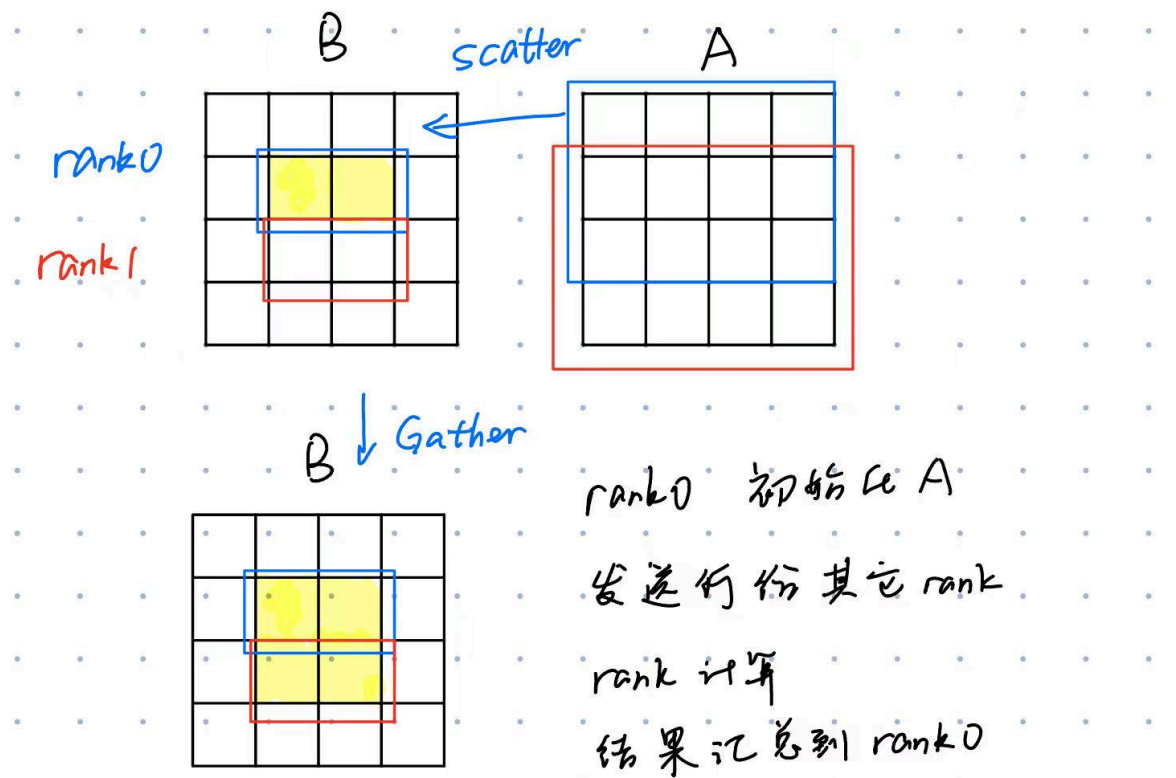
```
    for( j=1; j<N-1; j++)
```

```
        B[i][j] = (A[i-1][j] + A[i][j+1] + A[i+1][j] + A[i][j-1]) / 4.0
```

题中程序实现了如下功能：计算矩阵A除边界行其它所有位置的上下左右四个值的平均值，并存储在B矩阵的对应位置。

(1) 按行划分

主进程初始化矩阵A，按行划分发送给各个进程，完成计算后汇总到主进程。具体使用集群通信函数MPI_scatter和gather来实现，每个处理器读取目标行及上下两行的值。



// 分配矩阵A到每个处理器

```
int rows_per_proc = (N-2) / size;
double *local_A = (double *)malloc((rows_per_proc+2) * N * sizeof(double));
double *local_B = (double *)malloc(rows_per_proc * N * sizeof(double));
for (int i = 0; i < size; i++) {
    sendCount[i] = (rows_per_proc+2) * N; //发送目标行及其上下两行给各个进程
    offset[i] = 1 + rows_per_proc * i; //指定发送缓冲区中每个进程的偏移量
}
MPI_Scatterv(A, sendCount, offset, MPI_DOUBLE, local_A, (rows_per_proc+2)*N
```

// 数据收集

```
MPI_Gather(local_B, rows_per_proc * N, MPI_DOUBLE, B + N, rows_per_proc
```

运行结果如下：

```

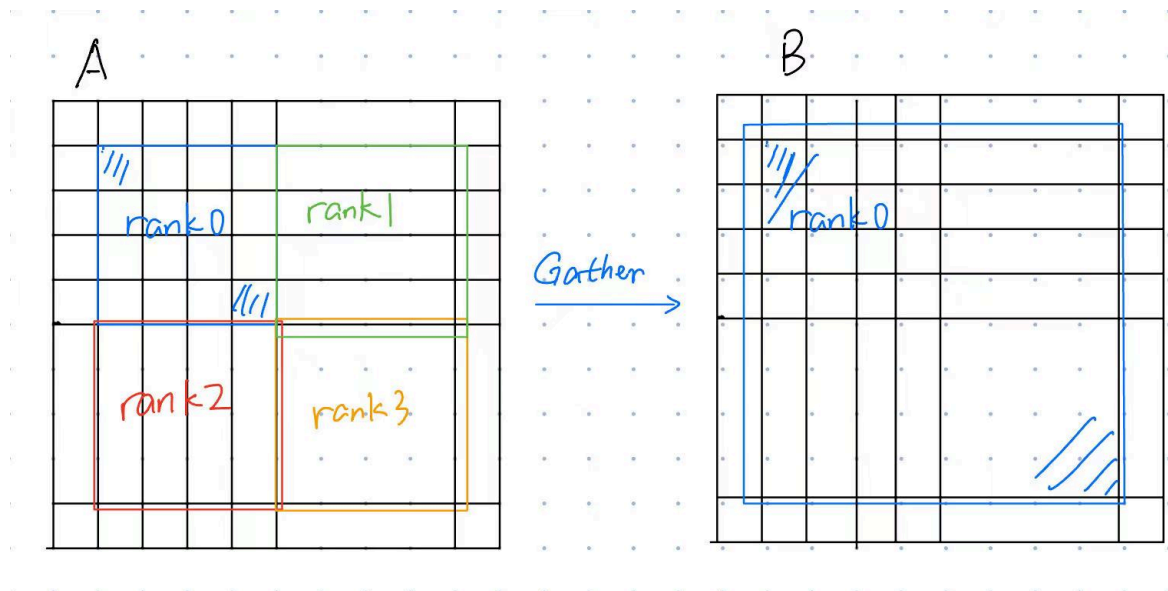
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pf> mpiexec -n 8 pf
main time:0.000811
Matrix A:
0 1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
2 3 4 5 6 7 8 9
3 4 5 6 7 8 9 10
4 5 6 7 8 9 10 11
5 6 7 8 9 10 11 12
6 7 8 9 10 11 12 13
7 8 9 10 11 12 13 14
Matrix B:
0 1 2 3 3 4 5 0
0 2 3 4 5 6 7 0
0 3 4 5 6 7 8 0
0 4 5 6 7 8 9 0
0 5 6 7 8 9 10 0
0 6 7 8 9 10 11 0
0 7 8 9 10 11 12 0
0 6 7 7 8 9 10 0

```

可知实现了程序的目标效果，计算正确。

(2) 棋盘式划分

每个进程初始化A和B，按块划分后计算结果汇总到主进程。



运行结果如下:

```
PS E:\graduate1stCourse\parallel\repo\mpi-lab\pf> mpiexec -n 8 pf-2
Result matrix B:
0 0 0 0 0 0 0 0 0 0
0 2 3 4 5 6 7 8 9 0
0 3 4 5 6 7 8 9 10 0
0 4 5 6 7 8 9 10 11 0
0 5 6 7 8 9 10 11 12 0
0 6 7 8 9 10 11 12 13 0
0 7 8 9 10 11 12 13 14 0
0 8 9 10 11 12 13 14 15 0
0 9 10 11 12 13 14 15 16 0
0 7 8 9 10 10 11 12 13 0
```